because this reduces cache usage and the chance of false sharing. If you have each thread handle a set of columns, it needs to read every value from the first matrix and the values from the corresponding columns in the second matrix, but you only have to write the column values. Given that the matrices are stored with the rows contiguous, this means that you're accessing *N* elements from the first row, *N* elements from the second, and so forth (where *N* is the number of columns you're processing). Since other threads will be accessing the other elements of each row, it's clear that you ought to be accessing adjacent columns, so the *N* elements from each row are adjacent, and you minimize false sharing. Of course, if the space occupied by your *N* elements is an exact number of cache lines, there'll be no false sharing because threads will be working on separate cache lines.

On the other hand, if you have each thread handle a set of *rows*, then it needs to read every value from the *second* matrix and the values from the corresponding *rows* of the *first* matrix, but it only has to write the row values. Because the matrices are stored with the rows contiguous, you're now accessing *all* elements from *N* rows. If you again choose adjacent rows, this means that the thread is now the *only* thread writing to those *N* rows; it has a contiguous block of memory that's not touched by any other thread. This is likely an improvement over having each thread handle a set of columns, because the only possibility of false sharing is for the last few elements of one block with the first few of the next, but it's worth timing it on the target architecture to confirm.

What about your third option—dividing into rectangular blocks? This can be viewed as dividing into columns and then dividing into rows. As such, it has the same false-sharing potential as division by columns. If you can choose the number of columns in the block to avoid this possibility, there's an advantage to rectangular division from the *read* side: you don't need to read the entirety of either source matrix. You only need to read the values corresponding to the rows and columns of the target rectangle. To look at this in concrete terms, consider multiplying two matrices that have 1,000 rows and 1,000 columns. That's 1 million elements. If you have 100 processors, they can handle 10 rows each for a nice round 10,000 elements. However, to calculate the results of those 10,000 elements, they need to access the entirety of the second matrix (1 million elements) plus the 10,000